



# Table des matieres

---

## 1. Introduction

- 1.1 Contexte et objectifs
- 1.2 Contraintes du projet

## 2. Architecture globale

- 2.1 Vue d'ensemble
- 2.2 Flux de donnees
- 2.3 Choix techniques

## 3. Apache Kafka - Mode KRaft

- 3.1 Architecture KRaft
- 3.2 Topics et partitionnement

## 4. Service Ingester

- 4.1 Sources WebSocket
- 4.2 Normalisation des donnees
- 4.3 Strategie de reconnexion

## 5. Service Processor - Analytique

- 5.1 VWAP
- 5.2 SMA-20
- 5.3 Detection d'anomalies Z-Score

## 6. Service API

- 6.1 Endpoints REST
- 6.2 Evenements Socket.IO

## 7. Dashboard

- 7.1 Stack frontend
- 7.2 Graphiques Chart.js

## 8. Securite

## 9. Multilinguisme - Module i18n

## 10. Deploiement Docker Compose

## 11. Tests et validation

## 12. Bilan et perspectives

## 13. Conclusion

# 1. Introduction

---

## 1.1 Contexte et objectifs

Les marches des cryptomonnaies sont caracterises par une volatilite extreme et des volumes de transactions considerables, generes 24h/24 par des milliers d'acteurs repartis dans le monde. Cette realite rend indispensable la mise en place d'outils de surveillance capables de traiter les donnees en temps reel, de calculer des indicateurs financiers pertinents et de detecter automatiquement les comportements anormaux du marche.

Le projet Crypto Market Monitor repond a ce besoin en implementant un pipeline de traitement de donnees en temps reel complet, depuis la collecte des flux WebSocket jusqu'a la visualisation sur un dashboard interactif. L'objectif pedagogique est de mettre en pratique les concepts fondamentaux du Real-Time Engineering: ingestion de flux, bus de messages, traitement en continu et interface utilisateur reactive.

## 1.2 Contraintes du projet

Ce projet a ete realise dans le cadre du module Real-Time Engineering de la formation M1 Data Engineering & IA a l'EFREI Paris. La contrainte principale a ete de mener a bien un projet prevu pour une equipe de 4 personnes avec seulement 2 etudiants: Adam Beloucif et Emilien Morice. Cette contrainte a necessite des choix architecturaux pragmatiques tout en maintenant un niveau de qualite professionnel.

- Equipe reduite: 2 personnes sur un projet calibre pour 4
- Stack imposee: Apache Kafka obligatoire comme bus de messages
- TypeScript: tout le code Node.js en TypeScript strict
- Deploiement Docker: chaque service dockerise avec healthchecks
- Temps reel: latence end-to-end inferieure a 500 ms

## 2. Architecture globale

### 2.1 Vue d'ensemble

L'architecture suit un pattern pipeline en étoile, avec Kafka comme backbone central qui découple les producteurs (ingester) des consommateurs (processor, API). Chaque service est indépendant, deployable séparément et communique exclusivement via les topics Kafka ou les interfaces HTTP/WebSocket exposées.



Figure 1 - Architecture pipeline Crypto Market Monitor

### 2.2 Flux de données

**Etape 1 - Collecte:** Les échanges Binance (BTC/USDT, ETH/USDT) et Coinbase (BTC-USD) publient des trades en continu via WebSocket. Chaque message contient le prix, la quantité échangée et l'horodatage.

**Etape 2 - Normalisation:** Le service Ingester traduit les formats propriétaires de chaque échange vers une interface Trade unifiée (symbol, price, qty, ts, exchange) avant de publier dans le topic Kafka crypto-trades.

**Etape 3 - Calcul analytique:** Le Processor consomme crypto-trades, maintient des buffers circulaires par symbole et calcule en continu VWAP, SMA-20, z-score et variations de prix. Les résultats sont publiés

dans crypto-metrics.

**Etape 4 - Distribution:** L'API Express consomme crypto-metrics, maintient un ring buffer de 500 entrees par symbole en memoire et pousse les mises a jour via Socket.IO a tous les clients connectes qui ont souscrit au symbole correspondant.

**Etape 5 - Visualisation:** Le Dashboard HTML/CSS/JS reçoit les donnees en temps reel, met a jour les graphiques Chart.js et declenche des alertes visuelles et sonores en cas d'anomalie.

## 2.3 Choix techniques

| Composant        | Technologie choisie     | Justification                                            |
|------------------|-------------------------|----------------------------------------------------------|
| Bus de messages  | Apache Kafka KRaft      | Impose par le cahier des charges, robustesse, decoupling |
| Runtime          | Node.js 20 + TypeScript | Typage fort, ecosysteme riche, I/O async natif           |
| WebSocket client | ws (npm)                | Legere, fiable, reconnexion maitrisable                  |
| Kafka client     | kafkajs                 | Mature, TypeScript-first, retry config fine              |
| API              | Express 4 + Socket.IO 4 | Standard industrie, push facile via Socket.IO            |
| Frontend         | HTML/CSS/JS vanilla     | Zero build step, bundle minimal, maintenable             |
| Graphiques       | Chart.js 4              | Performant, declaratif, mise a jour incrementale         |
| Conteneurisation | Docker Compose          | Reproductibilite, isolation, healthchecks                |

Tableau 1 - Choix technologiques

## 3. Apache Kafka - Mode KRaft

### 3.1 Architecture KRaft

Kafka 3.x introduit le mode KRaft (Kafka Raft Metadata Mode), qui integre la gestion des metadonnees directement dans le broker Kafka, eliminant ainsi la dependance historique a Apache Zookeeper. Dans ce projet, Kafka est configure en mode KRaft single-node, ce qui simplifie considerablement le deploiement Docker tout en conservant toutes les fonctionnalites necessaires pour un environnement de developpement.

Le broker joue simultanement les roles de controller et de broker, ce qui est acceptable en contexte de developpement ou de charge moderee. En production, une configuration multi-broker avec quorum Raft dedie serait recommandee.

### 3.2 Topics et partitionnement

| Topic          | Partitions | Retention | Replication | Usage                                     |
|----------------|------------|-----------|-------------|-------------------------------------------|
| crypto-trades  | 3          | 7 jours   | 1           | Trades bruts normalises depuis l'ingester |
| crypto-metrics | 3          | 1 jour    | 1           | Metriques calculees par le processor      |

Tableau 2 - Configuration des topics Kafka

#### Partitionnement par symbole

Les messages sont partitionnes par cle (symbol). Cette strategie garantit que tous les trades d'un meme symbole (ex: BTC/USDT) sont traites sequentiellement par le meme processor, preservant ainsi l'ordre necessaire aux calculs de series temporelles (VWAP, SMA).

#### Configuration Docker

```
# Variables d'environnement Kafka (docker-compose.yml)
KAFKA_NODE_ID: 1
KAFKA_PROCESS_ROLES: broker,controller
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
KAFKA_AUTO_CREATE_TOPICS_ENABLE: true
KAFKA_LOG_RETENTION_HOURS: 168
KAFKA_NUM_PARTITIONS: 3
```

## 4. Service Ingestor

### 4.1 Sources WebSocket

Le service Ingestor maintient simultanément trois connexions WebSocket persistantes vers les échanges Binance et Coinbase. Chaque échange a ses propres spécificités de protocole, notamment le format des messages et la méthode de souscription aux streams.

| Exchange | URL WebSocket                        | Symboles                     | Format        |
|----------|--------------------------------------|------------------------------|---------------|
| Binance  | wss://stream.binance.com:9443/ws     | btcusdt@trade, ethusdt@trade | JSON aggtrade |
| Coinbase | wss://advanced-trade-ws.coinbase.com | BTC-USD                      | JSON match    |

Tableau 3 - Sources de données WebSocket

### 4.2 Normalisation des données

Chaque échange publie ses trades dans un format JSON propre. Le service Ingestor normalise ces messages hétérogènes vers une interface Trade unifiée avant toute publication vers Kafka:

```
interface Trade {  
  symbol: string // ex: BTC/USDT  
  price: number // prix en USD  
  qty: number // quantité échangée  
  ts: number // timestamp en ms (Unix epoch)  
  exchange: string // 'binance' | 'coinbase'  
}
```

La normalisation inclut: conversion des champs string vers number (parseFloat), harmonisation des symboles (BTC-USD -> BTC/USDT), et horodatage normalisé en millisecondes.

### 4.3 Stratégie de reconnexion

Les connexions WebSocket peuvent être interrompues pour diverses raisons: problèmes réseau transitoires, maintenance des échanges, timeouts. Une stratégie de reconnexion robuste est indispensable pour un pipeline de production. Le service Ingestor implémente un backoff exponentiel avec jitter:

```
// Formule de backoff exponentiel avec jitter  
const delay = Math.min(  
  BASE_DELAY * Math.pow(2, attempt),  
  MAX_DELAY  
  ) + Math.random() * JITTER  
  
// Paramètres  
const BASE_DELAY = 1000 // 1 seconde  
const MAX_DELAY = 30000 // 30 secondes  
const JITTER = 1000 // +/- 1 seconde aléatoire
```

L'ajout du jitter (variation aléatoire) est crucial pour éviter le "thundering herd" problem: si plusieurs connexions tombent simultanément (ex: redémarrage du serveur), le jitter étale les tentatives de

reconnexion dans le temps, évitant une surcharge soudaine.



## 5. Service Processor - Analytique Temps Reel

Le service Processor est le coeur analytique du pipeline. Il consomme les trades bruts depuis le topic Kafka crypto-trades, maintient des structures de donnees en memoire (buffers circulaires) par symbole, et calcule en continu trois categories d'indicateurs: indicateurs de prix, indicateurs de volume, et detection d'anomalies.

### 5.1 VWAP - Volume Weighted Average Price

Le VWAP est un indicateur fondamental en trading haute frequence. Il represente le prix moyen pondere par le volume, offrant une mesure plus representative que la simple moyenne arithmetique des prix.

```
// Formule VWAP sur les N derniers trades
VWAP = SUM(price_i * qty_i, i=1..N) / SUM(qty_i, i=1..N)

// Implementation avec buffer circulaire (N = 100)
let sumPV = 0 // somme price * volume
let sumV = 0 // somme volume
buffer.getAll().forEach(t => {
  sumPV += t.price * t.qty
  sumV += t.qty
})
const vwap = sumV > 0 ? sumPV / sumV : 0
```

La fenetre glissante de 100 trades offre un bon compromis entre reactivite et stabilite. Le recalcul a chaque nouveau trade est  $O(N)$  avec le buffer circulaire, ce qui reste acceptable pour  $N=100$  et les frequences de trades observees.

### 5.2 SMA-20 - Moyenne Mobile Simple

La moyenne mobile simple sur 20 periodes lisse les fluctuations de court terme et permet de visualiser la tendance directionnelle du prix.

```
// Formule SMA sur N periodes
SMA_N = (1/N) * SUM(price_i, i=t-N+1 .. t)

// Implementation
const prices = buffer.getAll().slice(-20).map(t => t.price)
const sma20 = prices.reduce((a, b) => a + b, 0) / prices.length
```

### 5.3 Detection d'anomalies - Z-Score

Le z-score (score standardise) mesure l'ecart d'une valeur par rapport a la moyenne en unites d'ecart-type. Un z-score eleve en valeur absolue indique un mouvement statistiquement anormal du prix.

```
// Formule Z-Score
z = (x - mean) / std

// Avec:
// x = prix actuel
```

```
// mean = moyenne des 50 derniers prix
// std = ecart-type des 50 derniers prix

// Seuil d'anomalie: |z| > 2.5
const isAnomaly = Math.abs(zScore) > ANOMALY_THRESHOLD // 2.5

// Calcul de std
const mean = prices.reduce((a, b) => a + b) / prices.length
const variance = prices.reduce((s, p) => s + (p - mean)**2, 0) / prices.length
const std = Math.sqrt(variance)
```

Un seuil de 2.5 sigma signifie qu'environ 1.24% des variations de prix seront flaggées comme anomalies dans une distribution normale. Ce niveau est suffisamment sensible pour détecter les mouvements significatifs sans générer trop de faux positifs sur les marchés crypto intrinsèquement volatils.

## 6. Service API

Le service API est le point d'entree du frontend. Il encapsule la logique de consommation Kafka, maintient les ring buffers en memoire et gere les connexions Socket.IO entrantes des clients dashboard.

### 6.1 Endpoints REST

| Methode | Route                | Description               | Response              |
|---------|----------------------|---------------------------|-----------------------|
| GET     | /api/health          | Healthcheck               | { status: "ok", ts }  |
| GET     | /api/symbols         | Liste des symboles actifs | { symbols: string[] } |
| GET     | /api/metrics/:symbol | Dernieres metriques       | Metrics object        |
| GET     | /api/history/:symbol | Historique ring buffer    | Metrics[] (500 max)   |
| GET     | /api/anomalies       | Alertes recentes          | Anomaly[]             |

Tableau 4 - Endpoints REST de l'API

### 6.2 Evenements Socket.IO

| Evenement   | Direction         | Payload             | Description              |
|-------------|-------------------|---------------------|--------------------------|
| subscribe   | Client -> Serveur | { symbol: string }  | Abonnement a un symbole  |
| unsubscribe | Client -> Serveur | { symbol: string }  | Desabonnement            |
| metrics     | Serveur -> Client | Metrics object      | Mise a jour temps reel   |
| anomaly     | Serveur -> Client | Anomaly object      | Alerte anomalie detectee |
| error       | Serveur -> Client | { message: string } | Erreur de traitement     |

Tableau 5 - Evenements Socket.IO

### 6.3 Ring Buffer en memoire

Pour servir l'historique des metriques sans persistance base de donnees, chaque symbole dispose d'un buffer circulaire de 500 entrees maintenu en memoire vive. Cette approche offre des lectures  $O(1)$  et une empreinte memoire constante.

```
// Ring buffer par symbole
const ringBuffers = new Map<string, CircularBuffer<Metrics>>()

// A chaque message Kafka
consumer.on('metrics', (metrics: Metrics) => {
  if (!ringBuffers.has(metrics.symbol)) {
    ringBuffers.set(metrics.symbol, new CircularBuffer(500))
  }
  ringBuffers.get(metrics.symbol)!.push(metrics)
  io.to(metrics.symbol).emit('metrics', metrics)
})
```

## 7. Dashboard

### 7.1 Stack frontend

Le dashboard est volontairement developpe en HTML/CSS/JS vanilla sans framework, conformement aux contraintes du module. Cette approche minimise la complexite du build et offre un temps de chargement optimal. La communication temps reel est assuree par Socket.IO Client, et les graphiques par Chart.js.

| Technologie       | Version | Usage                                                        |
|-------------------|---------|--------------------------------------------------------------|
| HTML5/CSS3        | N/A     | Structure et styles, CSS Grid/Flex, CSS Variables dark theme |
| JavaScript ES2022 | vanilla | Logique applicative, gestion evenements Socket.IO            |
| Chart.js          | 4.x     | Graphiques prix (line) et volume (bar), mise a jour live     |
| Socket.IO Client  | 4.x     | Reception push des metriques depuis l'API                    |
| Web Audio API     | native  | Beep sonore sur detection anomalie (AudioContext)            |
| nginx             | alpine  | Serveur statique Docker pour les fichiers HTML/CSS/JS        |

Tableau 6 - Stack frontend dashboard

### 7.2 Graphiques Chart.js

Le dashboard affiche deux types de graphiques mis a jour en temps reel:

- Graphique de prix (type line): courbe des 100 derniers prix, mise a jour par ajout/suppression du premier point sans re-rendu complet (`chart.data.labels.shift() + push()`)
- Graphique de volume (type bar): volume des 20 derniers trades, couleur verte/rouge selon variation
- Indicateurs VWAP et SMA-20 superposes sur le graphique de prix comme lignes de reference

### 7.3 Alertes anomalies

Lorsque le Processor detecte un z-score superieur a 2.5, l'evenement anomaly est emis via Socket.IO. Le dashboard reagit en:

- Ajoutant une entree rouge dans le flux d'alertes avec horodatage, symbole et z-score
- Jouant un beep sonore via AudioContext (frequence 880 Hz, duree 200 ms)
- Affichant un badge rouge clignotant sur le symbole concerne

## 8. Securite

La securite est integree a tous les niveaux du pipeline. Le service API, en tant que point d'entree public, concentre la majorite des mesures de protection.

| Categorie     | Mesure                | Implementation                                          |
|---------------|-----------------------|---------------------------------------------------------|
| HTTP Headers  | helmet                | CSP, HSTS, X-Frame-Options, X-Content-Type-Options      |
| Rate Limiting | express-rate-limit    | 100 requetes/min par IP, WindowMs: 60s                  |
| CORS          | Configuration stricte | Origines whitelist, methodes autorisees explicites      |
| Validation    | Zod schemas           | Validation des variables d'env au demarrage (fast-fail) |
| Secrets       | .env + gitignore      | Aucun secret en dur dans le code source                 |
| Docker        | USER non-root         | Containers executes avec user node:node (UID 1000)      |
| Images        | Alpine/slim           | Surface d'attaque minimale, pas d'outils inutiles       |
| Erreurs       | Masquage              | Stack traces jamais exposees au client HTTP             |
| Socket.IO     | Namespace validation  | Validation des symboles a la souscription               |

Tableau 7 - Mesures de securite implementees

### Validation Zod des variables d'environnement

```
import { z } from 'zod'

const envSchema = z.object({
  KAFKA_BROKERS: z.string().min(1),
  PORT: z.coerce.number().default(3000),
  CORS_ORIGINS: z.string().default('http://localhost:8000'),
  NODE_ENV: z.enum(['development', 'production']).default('development'),
})

// Echec immediat si variable manquante ou invalide
export const env = envSchema.parse(process.env)
```

## 9. Multilinguisme - Module i18n

Le dashboard integre un module d'internationalisation (i18n) en JavaScript vanilla qui permet de basculer entre le francais et l'anglais sans rechargement de page. La langue choisie est persistee dans localStorage.

### Architecture du module

```
// dashboard/js/i18n.js
const translations = {
  fr: {
    title: 'Surveillance Crypto',
    price: 'Prix actuel',
    anomaly: 'Anomalie detectee',
    // ... 30+ cles
  },
  en: {
    title: 'Crypto Monitor',
    price: 'Current price',
    anomaly: 'Anomaly detected',
    // ...
  }
}

function setLanguage(lang) {
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.dataset.i18n
    el.textContent = translations[lang]?.[key] ?? translations.fr[key]
  })
  localStorage.setItem('lang', lang)
}
```

### Utilisation dans le HTML

```
<!-- Attribut data-i18n sur les elements a traduire -->
<h2 data-i18n="price">Prix actuel</h2>
<span data-i18n="anomaly">Anomalie detectee</span>
<button onclick="setLanguage('en')" data-i18n="lang_toggle">EN</button>
```

| Cle     | Francais            | Anglais           |
|---------|---------------------|-------------------|
| title   | Surveillance Crypto | Crypto Monitor    |
| price   | Prix actuel         | Current price     |
| volume  | Volume              | Volume            |
| anomaly | Anomalie detectee   | Anomaly detected  |
| vwap    | VWAP                | VWAP              |
| sma     | Moyenne mobile 20   | Moving average 20 |

|              |                 |              |
|--------------|-----------------|--------------|
| change1m     | Variation 1 min | 1 min change |
| change5m     | Variation 5 min | 5 min change |
| connected    | Connecte        | Connected    |
| disconnected | Deconnecte      | Disconnected |

Tableau 8 - Extrait des cles de traduction

## 10. Deploiement Docker Compose

Le projet est entierement conteneurise via Docker Compose. Six services cooperent via un reseau bridge interne, avec des healthchecks pour garantir l'ordre de demarrage et la disponibilite de chaque composant.

### Services Docker Compose

| Service   | Image                     | Port expose | Dependances | Role                 |
|-----------|---------------------------|-------------|-------------|----------------------|
| kafka     | confluentinc/cp-kafka:7.6 | 9092        | aucune      | Broker Kafka KRaft   |
| kafka-ui  | provectus/kafka-ui        | 8080        | kafka       | Interface monitoring |
| ingester  | node:20-alpine            | aucun       | kafka       | Collecte WebSocket   |
| processor | node:20-alpine            | aucun       | kafka       | Calcul analytique    |
| api       | node:20-alpine            | 3000        | kafka       | API REST + Socket.IO |
| dashboard | nginx:alpine              | 8000        | api         | Serveur statique     |

Tableau 9 - Services Docker Compose

### Variables d'environnement

| Variable        | Service                  | Valeur par default                   | Description               |
|-----------------|--------------------------|--------------------------------------|---------------------------|
| KAFKA_BROKERS   | ingester, processor, api | kafka:9092                           | Adresse du broker Kafka   |
| BINANCE_WS_URL  | ingester                 | wss://stream.binance.com:9443/ws     | URL WebSocket Binance     |
| COINBASE_WS_URL | ingester                 | wss://advanced-trade-ws.coinbase.com | URL WebSocket Coinbase    |
| PORT            | api                      | 3000                                 | Port d'ecoute de l'API    |
| CORS_ORIGINS    | api                      | http://localhost:8000                | Origines CORS autorisees  |
| NODE_ENV        | tous                     | development                          | Environnement d'execution |

Tableau 10 - Variables d'environnement

### Commandes de gestion

```
# Demarrage de l'ensemble du stack
docker compose up -d

# Suivi des logs en temps reel
docker compose logs -f

# Logs d'un service specifique
docker compose logs -f ingester

# Arret et suppression des volumes
docker compose down -v
```



```
# Acces aux interfaces  
# Dashboard: http://localhost:8000  
# API REST: http://localhost:3000/api/health  
# Kafka UI: http://localhost:8080
```

## 11. Tests et validation

La strategie de tests du projet combine des validations manuelles des flux end-to-end et des verifications unitaires des formules analytiques. La nature temps reel du systeme rend les tests d'integration automatises plus complexes mais non impossibles a mettre en place pour une version future.

### Scenarios de test manuels

| Scenario              | Etapes                                                     | Resultat attendu                                                |
|-----------------------|------------------------------------------------------------|-----------------------------------------------------------------|
| Demarrage complet     | docker compose up -d, attendre 30s, ou Dashboard           | Dashboard affiche prix BTC/USDT, ETH/USDT en temps reel         |
| Detection anomalie    | Monitorer le flux pendant un mouvement de marche - violent | Alerte rouge visible dans le dashboard, event anomaly Socket.IO |
| Reconnexion WebSocket | Couper reseau 10s, retablir                                | Ingesteur se reconnecte automatiquement, flux reprend           |
| Kafka UI              | Ouvrir localhost:8080, inspecter topics                    | crypto-trades et crypto-metrics visibles avec messages actifs   |
| Changement de langue  | Cliquer bouton EN sur le dashboard                         | Labels UI basculent en anglais sans rechargement                |
| Rate limiting API     | Envoyer > 100 req/min vers /api/health                     | HTTP 429 Too Many Requests retourne                             |
| Health endpoints      | GET /api/health, GET /api/symbols                          | Reponses JSON valides avec donnees actuelles                    |

Tableau 11 - Scenarios de tests manuels

### Validation des formules analytiques

Les formules VWAP, SMA-20 et Z-Score ont ete validees manuellement sur des series de prix connues:

- VWAP: verifie sur une serie de 5 trades avec volumes variables, resultat compare a calcul Excel
- SMA-20: verifie contre les valeurs publiees par TradingView sur BTC/USDT
- Z-Score: verifie sur une distribution normale artificielle (seuil 2.5 = 1.24% outliers attendus)

## 12. Bilan et perspectives

### Ce qui a été réalisé

- Pipeline temps réel complet et fonctionnel: WebSocket -> Kafka -> Analytics -> Dashboard
- Kafka KRaft sans Zookeeper: simplification du déploiement Docker
- 3 feeds WebSocket: Binance (BTC/USDT, ETH/USDT) et Coinbase (BTC-USD)
- 3 indicateurs analytiques: VWAP, SMA-20, Z-Score avec détection d'anomalies
- Dashboard live avec Chart.js, alertes visuelles et sonores
- Module i18n FR/EN en JavaScript vanilla, persistance localStorage
- Sécurité: helmet, rate-limit, Zod validation, CORS, containers non-root
- Déploiement Docker Compose 6 services avec healthchecks et volumes nommés
- Projet réalisé par 2 personnes (prévu pour 4) avec qualité professionnelle

### Perspectives d'évolution

- Persistance long terme: TimescaleDB ou InfluxDB pour historique sur plusieurs jours
- Alertes push: notifications Telegram ou email sur détection d'anomalie critique
- Machine Learning: prédiction de tendance avec modèle LSTM ou Prophet
- Exchanges supplémentaires: Kraken, OKX pour diversifier les sources de prix
- Authentification: JWT + gestion de portefeuilles personnalisés
- Backtest: capacité de replay des données historiques sur le pipeline analytique
- CI/CD: GitHub Actions avec tests automatisés E2E (Playwright) et lint TypeScript
- Observabilité: Prometheus + Grafana pour métriques infra (latence, throughput, consumer lag)
- Clustering Kafka: passage à 3 brokers pour haute disponibilité

### Retour d'expérience

Ce projet a permis d'acquérir une expérience pratique significative sur plusieurs technologies de l'écosystème Real-Time Engineering:

| Domaine               | Compétences acquises                                                     |
|-----------------------|--------------------------------------------------------------------------|
| Apache Kafka          | Configuration KRaft, topics, partitionnement, consumer groups, kafkajs   |
| WebSocket             | Gestion de connexions persistantes, protocoles Binance/Coinbase, backoff |
| Analytique temps réel | VWAP, SMA, Z-score, buffers circulaires, algorithmes O(1)                |
| TypeScript            | Type strict, interfaces, generics, enums, schema Zod                     |
| Socket.IO             | Rooms/namespaces, push server-side, reconnexion client                   |
| Docker                | Compose multi-service, healthchecks, volumes, réseau bridge, non-root    |

|              |                                                                 |
|--------------|-----------------------------------------------------------------|
| Securite API | Helmet, rate-limiting, CORS, validation env, secrets management |
|--------------|-----------------------------------------------------------------|

Tableau 12 - Competences acquises

## 13. Conclusion

---

Le projet Crypto Market Monitor constitue une demonstration concrete et fonctionnelle des concepts fondamentaux du Real-Time Engineering appliques au domaine financier. Le pipeline complet - de la collecte WebSocket a la visualisation sur dashboard - illustre comment Apache Kafka peut servir de backbone scalable pour des architectures evenementielles en temps reel.

La realisation de ce projet par une equipe de deux personnes, pour un cahier des charges prevu pour quatre, demontre la capacite a prioriser les fonctionnalites essentielles, a faire des choix architecturaux pragmatiques et a maintenir une qualite de code professionnelle sous contrainte de ressources.

Sur le plan technique, les principaux apports sont: la maitrise de Kafka KRaft comme bus de messages decoupled, l'implementation d'indicateurs financiers temps reel (VWAP, SMA, Z-score) avec des structures de donnees adaptees (buffers circulaires), et l'integration d'un stack full TypeScript Node.js containerise avec Docker Compose.

Les perspectives d'evolution sont nombreuses: persistance long terme avec TimescaleDB, alertes push multi-canal, modeles de prediction ML, et passage a une architecture Kafka multi-broker pour la haute disponibilite. Ces evolutions permettraient de transformer ce proof of concept en plateforme de surveillance professionnelle.

---

Adam Beloucif - Emilien Morice  
M1 Data Engineering & IA - EFREI Paris  
Module Real-Time Engineering - 2025-2026

